AXEL SPRINGER TECH
Quality Assurance · Mobile

QA Manager Take-Home Challenge

Cross-platform screenshot testing with mocked data, running against a production codebase across iOS and Android.

TIME BUDGET	~ until our appointment
FORMAT	Written strategy + a working demo (your choice of tooling)
APPLIES TO	BILD News · Sport BILD · WELT News · WELT Edition

THE ROLE

About this challenge

At Axel Springer Tech our corporate strategy puts AI at the centre: *“Digital is the new print. AI is the new digital.”* You would be joining the cross-functional team behind Germany’s leading news apps — BILD News, Sport BILD, WELT News and WELT Edition — reaching millions of users every month.

This exercise is a realistic slice of the work. We are not looking for a perfect, production-ready system in a day. We want to understand how you think about quality at scale: the strategy you choose, the trade-offs you weigh, how you make tests deterministic and trustworthy, and how you would roll this out and mentor a team around it. A rough-but-honest demo with clear reasoning beats a polished black box.

> **What we want to learn about you**
>
> - How you design a QA strategy end to end and justify tooling choices.
> - How you make UI tests **deterministic** by controlling data and environment.
> - How you think across **both platforms** without doubling the effort.
> - How you integrate testing into CI/CD and a Scrum delivery process.
> - How clearly you communicate — the way you would with developers, POs and designers.

CONTEXT

The scenario

Our news apps are visually rich and highly stateful. A single article feed renders breaking-news banners, live tickers, teaser cards with and without images, video teasers, ads, and premium (paywalled) items for WELT Apps. Every screen has to look right in light and dark mode, across dynamic font sizes, for very long German headlines, in offline and error states, and on a range of device sizes — on both iOS and Android.

Because these states depend on backend data, visual regressions are easy to miss: a spacing bug on the breaking-news banner, a truncated headline, a broken dark-mode contrast, or a paywall badge that disappears after a refactor. Reproducing every state by hand across two platforms before each release does not scale, and it slows the Scrum team down.

> **The problem to solve**
>
> We want a **screenshot (visual regression) testing** capability that renders real UI from the **production codebase**, feeds it **mocked, deterministic data** so every state is reproducible, and works for **both iOS and Android** — catching unintended visual changes automatically on every pull request.

YOUR TASK

Your mission

Design and demonstrate a cross-platform screenshot-testing approach for our apps. You choose the path — native tooling per platform, a single cross-platform tool, or a hybrid. Both of the following are in scope:

1. **A screenshot-testing strategy** (written, ~1–3 pages) covering both iOS and Android: your tool choice and why, how you inject mocked data to make snapshots deterministic, how golden/reference images are stored and reviewed, how it plugs into CI/CD, how you keep it stable (flakiness), and how it scales as the apps and team grow.
2. **A working demo** for **at least one** platform: screenshot tests for a few representative components or screens (e.g. a breaking-news banner, a teaser card, a paywalled item) driven by mocked data. Include your golden images and, ideally, one **deliberately introduced regression** so we can see the failing diff and your review workflow.

You do not need working code on both platforms. A solid demo on one, plus a written strategy that convincingly covers the other, is exactly right for the time budget.

STARTER REFERENCE

A worked example (so we share a definition)

To make sure we all mean the same thing by “screenshot testing with mocked data”, here is a minimal illustration on each platform. This is only a reference point — you are free to use entirely different tools. The essential idea is the same everywhere: **build the real view, inject a fixed data fixture, render it off-device, and compare the output to a committed reference image.**

iOS — swift-snapshot-testing (Point-Free)

ArticleCardSnapshotTests.swift

```
import SnapshotTesting
import XCTest
@testable import NewsApp

final class ArticleCardSnapshotTests: XCTestCase {
  func test_breakingNewsCard_lightAndDark() {
    // Deterministic fixture: no network, fixed date & assets
    let article = Article.mock(
      kind: .breaking,
      headline: "Bundestag beschliesst neues Gesetz",
      publishedAt: Date(timeIntervalSince1970: 1_700_000_000)
    )
    let view = ArticleCardView(viewModel: .init(article: article))

    assertSnapshot(of: view, as: .image(on: .iPhone13(.portrait)))
    assertSnapshot(of: view,
      as: .image(traits: .init(userInterfaceStyle: .dark)))
  }
}
```

Android — Paparazzi (Jetpack Compose)

ArticleCardScreenshotTest.kt

```
class ArticleCardScreenshotTest {
  @get:Rule val paparazzi = Paparazzi(deviceConfig = PIXEL_5)

  @Test fun breakingNewsCard_dark() {
    val article = mockArticle(              // deterministic fixture
      kind = Kind.BREAKING,
      headline = "Bundestag beschliesst neues Gesetz",
    )
    paparazzi.snapshot {
      NewsTheme(dark = true) { ArticleCard(article) }
    }
  }
}
```

Paparazzi and Roborazzi render Android UI on the JVM without an emulator, which keeps runs fast and deterministic; the iOS example renders the view in-process. Both commit a reference PNG and fail the build on any pixel difference beyond a configured threshold.

> **Prefer platform / cloud tooling instead of unit-level snapshots?**
> That is a perfectly valid direction. You might drive the running app with a UI-automation tool (e.g. Maestro, Appium or XCUITest / Espresso) and capture screenshots on simulators/emulators or on a device cloud (Sauce Labs, BrowserStack, Firebase Test Lab), then diff them with a visual service (e.g. Applitools or Emerge). In that case the mocked data usually comes from a **local mock server** (WireMock / embedded stub) or a launch-argument / build-flavour that swaps the data source for fixtures. Tell us why you chose it and how you keep it fast and stable.

WHAT TO HAND IN

Deliverables

1. **Strategy document** (PDF / Markdown, ~1–3 pages) — your approach for both platforms, tooling rationale, mocking strategy, golden management, CI/CD integration, flakiness handling, scaling and a short rollout plan.
2. **Demo project** (Git repo link or zip) — runnable screenshot tests on at least one platform, with mocked data and committed reference images. A deliberately introduced regression with its diff is a big plus.
3. **README** — how to run the demo and regenerate golden images, in a few clear commands.
4. **Also add which prompts you have used**.

5. *Optional* — a 3–5 minute screen recording or notes walking us through your reasoning.

> **You will not receive our real codebase**
> Build a small sample app (or adapt an open-source one) that mimics a couple of our screens — a feed with a breaking-news banner. In your strategy doc, describe how you would integrate the same approach into a large, existing production codebase: where fixtures live, how you seam the network layer for mocking, and how you would introduce it without disrupting the team.